

Tous les documents sont interdits. Tous les usages de dispositifs électroniques sont interdits, sauf évidemment ce qui est strictement requis en télé-examen, c'est-à-dire l'accès au sujet et l'envoi d'un scan de la copie papier. Par conséquent, tout usage de logiciel (éditeur de fichiers, compilateur, prouveur, etc) est interdit, même pour rédiger la copie d'examen. La copie d'examen doit être entièrement rédigée à la main.

Les règles de la logique de Hoare et la syntaxe de la logique du langage micro-C sont rappelées dans l'annexe à la fin de ce document. Un barème est donné à titre indicatif.

Exercice 1 : Carré en logique de Hoare (9 points)

Question 1 : Exprimer par un triplet de Hoare que, si le programme suivant termine, alors il calcule dans c le carré de tout nombre n positif ou nul.

```
 $c := 0; i := 0; \text{ while } i < n \text{ do } c := c + i; i := i + 1; c := c + i \text{ od}$ 
```

Question 2 : En appliquant une par une les règles de la logique de Hoare, démontrer formellement dans un tableau que ce triplet de Hoare est valide.

Question 3 : Définir une fonction C

```
int square(int n)
```

qui implémente le programme de ce triplet de Hoare et retourne la valeur qu'il calcule dans sa variable c .

Question 4 : Transformer la précondition et la postcondition de ce triplet de Hoare en annotations micro-C pour cette fonction.

Question 5 : Proposer un invariant micro-C assez précis pour prouver cette postcondition.

Question 6 : Proposer un variant micro-C permettant de prouver la terminaison de cette fonction C.

Exercice 2 : Formules sur les entiers (5 points)

Le but de cet exercice est de formaliser par des lemmes micro-C des propriétés de parité des entiers. Certains lemmes peuvent aider à en démontrer d'autres.

Question 1 : Définir en syntaxe micro-C deux prédicats `even` et `odd` indiquant respectivement si un entier est pair ou impair, en utilisant seulement la logique du premier ordre, l'addition, la multiplication et l'égalité.

Question 2 : Formaliser chacune des propriétés suivantes par un lemme dans la logique de micro-C.

1. Tout entier s'écrivant comme le double d'un autre entier est pair.
2. Tout entier de la forme $2k + 2$ est pair.
3. Tout entier pair n'est pas impair.
4. Le successeur de tout entier pair est impair.

Question 3 : En n'utilisant que la logique du premier ordre, la multiplication et l'égalité, définir en micro-C un prédicat `divides` indiquant qu'un entier d divise un entier n .

Question 4 : Formaliser la propriété suivante par un lemme dans la logique de micro-C : *Un entier est pair si et seulement s'il est divisible par 2.*

Exercice 3 : Tableau aléatoire décroissant (6 points)

On rappelle que l'expression `rand()` fournit un entier pseudo-aléatoire positif ou nul. Le programme C suivant remplit tout tableau d'entiers `t` avec des nombres aléatoires **strictement décroissants**.

```
void random_decreasing_array(int t[], int n) {
    int i = 0;
    int v = 0;
    while (i < n) {
        v = v - rand() - 1;
        t[i] = v;
        i = i + 1;
    }
}
```

Question 1 : Spécifier par une précondition micro-C que le paramètre `n` de cette fonction est la longueur du tableau `t`.

Question 2 : Pour cette fonction, proposer une postcondition micro-C qui décrit que le tableau résultant est strictement décroissant.

Question 3 : Dans un invariant de boucle en syntaxe micro-C, spécifier les bornes les plus précises possible pour la variable `i`.

Question 4 : Pour `i` non nul, spécifier l'invariant de boucle le plus précis possible pour la valeur de la variable `v`.

Question 5 : Spécifier un invariant de boucle assez précis pour permettre de démontrer automatiquement la postcondition de la question 2 (avec l'aide des invariants précédents).

Question 6 : Spécifier un invariant de boucle permettant de démontrer la terminaison de cette fonction.

Annexe : rappels de cours

Logique de Hoare

La logique de Hoare est le système formel $\langle L, R \rangle$ où

- L est l'ensemble des formules du premier ordre et des formules de la forme $\{P\} c \{Q\}$, appelées triplets de Hoare, où P et Q sont des formules de la logique du premier ordre et c est un programme du mini-langage défini par la grammaire

$c, c' ::=$	$ x := e$	affectation
	$ c; c'$	séquence
	$ \text{if } b \text{ then } c \text{ fi}$	conditionnelle simple
	$ \text{if } b \text{ then } c \text{ else } c' \text{ fi}$	conditionnelle double
	$ \text{while } b \text{ do } c \text{ od}$	répétition
- R est l'ensemble de règles de déduction de la figure 1.

$$\begin{array}{c}
 \frac{}{\{P (e/x)\} x := e \{P\}} \quad (\text{assignment}) \\
 \\
 \frac{\{P\} c \{Q\} \quad \{Q\} c' \{R\}}{\{P\} c; c' \{R\}} \quad (\text{sequence}) \\
 \\
 \frac{\{b \wedge P\} c \{Q\} \quad (\neg b \wedge P) \Rightarrow Q}{\{P\} \text{if } b \text{ then } c \text{ fi } \{Q\}} \quad (\text{if-then}) \\
 \\
 \frac{\{b \wedge P\} c_1 \{Q\} \quad \{\neg b \wedge P\} c_2 \{Q\}}{\{P\} \text{if } b \text{ then } c_1 \text{ else } c_2 \text{ fi } \{Q\}} \quad (\text{if-then-else}) \\
 \\
 \frac{\{b \wedge P\} c \{P\}}{\{P\} \text{while } b \text{ do } c \text{ od } \{\neg b \wedge P\}} \quad (\text{while}) \\
 \\
 \frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \quad (\text{consequence}) \\
 \\
 \frac{P \Rightarrow P' \quad \{P'\} c \{Q\}}{\{P\} c \{Q\}} \quad (\text{precondition}) \\
 \\
 \frac{\{P\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \quad (\text{postcondition})
 \end{array}$$

FIGURE 1 – Règles de la logique de Hoare.

Syntaxe de la logique du langage micro-C

La logique du langage micro-C doit être écrite dans un fichier C, soit sur une ligne qui commence par `//@`, soit dans un bloc de commentaires qui commence par `/*@` et se termine par `*/`.

Les types de micro-C sont les types C suivants :

- Le type `void`.
- Le type `int` des entiers relatifs des mathématiques, avec le symbole fonctionnel unaire `-` (préfixé) et les symboles fonctionnels binaires infixés `+`, `-`, `*`, `/` et `%`, respectivement pour l'opposé, l'addition, la soustraction, la multiplication, le quotient et le reste de la division euclidienne.

— Le type `int` .. `[]` des tableaux d'entiers.

La logique propositionnelle de micro-C contient les formules `true` et `false` pour “vrai” et “faux”. Les connecteurs propositionnels de la négation, du “et”, du “ou”, de l'implication et de l'équivalence sont respectivement `!`, `&&`, `||` (comme en C), `->` et `<->`.

La quantification universelle $\forall x...$ s'écrit `forall x. ..` si `x` est un entier et `forall x[]. ..` si `x` est un tableau d'entiers. De même pour la quantification existentielle. Le quantificateur existentiel $\exists$ s'écrit `exists`. Sur papier, on peut aussi utiliser les formes plus lisibles $\forall$ et $\exists$.

Le prédicat d'égalité infixé (`=`) est prédéfini sur les types des entiers et des tableaux d'entiers.

Sur le type `int`, les symboles relationnels binaires (infixés) `!=`, `<`, `>`, `<=` et `>=` sont aussi définis. Ils correspondent respectivement à la négation de l'égalité et aux relations d'ordre strictes et larges sur les entiers. Sur papier, on peut aussi écrire $\neq$, $<$, $>$, $\leq$ et $\geq$.

Dans la partie logique du langage micro-C le terme `length(a)` désigne la longueur du tableau d'entiers `a`. Comme en C, le terme `a[i]` désigne l'élément du tableau `a` à l'indice `i`, si cet indice est entre 0 et `length(a)-1` inclus.

La ligne

```
predicate p();
```

déclare une proposition atomique p , aussi appelée “variable propositionnelle”.

La ligne

```
predicate p(int i, int t[]);
```

déclare un prédicat binaire p dont le premier argument est un entier et le second argument est un tableau d'entiers.

La ligne

```
function T f(int i, int t[]);
```

déclare une fonction binaire f dont le premier argument est un entier, le second argument est un tableau d'entiers, et qui calcule une donnée de type `T`, qui peut être `void` ou `int`. Des axiomes (propriétés admises) de ce prédicat ou de cette fonction peuvent être énoncés par des lignes telles que

```
lemma axiomName: ..
```

où `axiomName` est un nom d'axiome à choisir et où les pointillés doivent être remplacés par une formule micro-C qui porte sur ce prédicat ou cette fonction.

Les lignes

```
predicate p(int i, int t[]) = ..;  
function T f(int i, int t[]) = ..;
```

déclarent un prédicat p et une fonction similaires, tout en donnant une définition à ce prédicat ou à cette fonction. Pour le prédicat, il faut remplacer les pointillés par une formule micro-C. Pour la fonction, il faut remplacer les pointillés par une expression de type `T`.

La ligne

```
lemma lemmaName: ..
```

déclare un lemme nommé `lemmaName`. Les pointillés doivent être remplacés par une formule micro-C.

Pour décrire en micro-C la valeur d'une expression **avant** exécution d'une fonction, il faut entourer cette expression par `old(` et `)`, dans toute postcondition ou invariant d'une boucle de cette fonction.